PLAN DE EXPOSICIÓN

Proyecto Backend II

API REST de gestión de estudiantes, cursos y matrículas

Integrantes	Santiago Arango, Alex Monroy, Jose Vasquez e Isaac
Duración	13-15 minutos más preguntas
Tecnologías	Java 21, Spring Boot 4, Spring Data JPA, Bean Validation, MySQL (XAMPP), Maven

Sobre el proyecto

El sistema permite registrar estudiantes, crear cursos con un cupo máximo y matricular estudiantes en esos cursos. Antes de guardar se aplican reglas de negocio: el correo de cada estudiante y el código de cada curso son únicos, un estudiante no puede tener dos matrículas vigentes en el mismo curso y no se puede matricular en un curso que ya llegó a su cupo. Los errores se responden con códigos HTTP claros (400, 404 y 409).

Orden de la exposición

#	Integrante	Tema	Tiempo
1	Santiago Arango	Introducción, tecnologías y arquitectura	3 min
2	Alex Monroy	Modelo de datos: entidades, relaciones y repositorios	3 min
3	Jose Vasquez	Lógica de negocio: servicios, reglas y excepciones	3-4 min
4	Isaac	API REST, validaciones, errores y demo en vivo	4-5 min

PARTE 1 · 3 MIN

Santiago Arango

Introducción, tecnologías y arquitectura

De qué habla

- **Qué es el proyecto.** API REST para gestionar estudiantes, cursos y matrículas, con reglas como correo único, cupo máximo por curso y no matricular dos veces a la misma persona.
- **Tecnologías.** Java 21, Spring Boot 4, Spring Web MVC, Spring Data JPA (Hibernate), Validation, MySQL con XAMPP y Maven.
- **Configuración.** application.yml: puerto 8888, conexión a MySQL y ddl-auto: update, que crea las tablas a partir de las clases.
- **Arquitectura por capas.** controller, dto, service, repository y domain, cada uno con una sola responsabilidad, y el recorrido de una petición de punta a punta.

Qué muestra

pom.xml · application.yml · Árbol de paquetes del proyecto

PARTE 2 · 3 MIN

Alex Monroy

Modelo de datos: entidades, relaciones y repositorios

De qué habla

- **Las tres entidades.** Student (estudiante), Course (curso) y Enrollment (matrícula), y cómo JPA las convierte en tablas con @Entity, @Table e @Id autoincremental.
- **Relaciones.** Student 1-N Enrollment N-1 Course. Por qué la matrícula es una entidad propia y no un @ManyToMany: guarda fecha y estado. @ManyToOne, @JoinColumn, mappedBy y cascade.
- **Estados de la matrícula.** Enum EnrollmentStatus (ACTIVE, CANCELLED, COMPLETE) guardado como texto con EnumType.STRING.
- **Repositorios.** JpaRepository da el CRUD sin escribir código, y las consultas derivadas del nombre del método (existsByEmail, countByCourse_IdAndStatus).

Qué muestra

Student.java · Enrollment.java · EnrollmentStatus.java · EnrollmentRepository.java · Tablas en phpMyAdmin (opcional)

PARTE 3 · 3-4 MIN

Jose Vasquez

Lógica de negocio: servicios, reglas y excepciones

De qué habla

- **Estructura de los servicios.** Interfaz + implementación (Impl), @Service, inyección por constructor y @Transactional (todo o nada).
- **Reglas de estudiantes y cursos.** Correo y código de curso únicos, y no se puede bajar el cupo de un curso por debajo de sus matrículas activas.
- **Reglas de matrícula.** Las cinco verificaciones al matricular: estudiante existe, curso existe, no está ya inscrito, hay cupo, y se guarda como ACTIVE.
- **Cambios de estado.** Qué transiciones se permiten entre ACTIVE, CANCELLED y COMPLETE.
- **Excepciones.** Jerarquía de excepciones propias: no encontrado (404), conflicto (409) y error de negocio (400).

Qué muestra

EnrollmentServiceImpl.java · StudentServiceImpl.java · Carpeta domain/Exceptions

PARTE 4 · 4-5 MIN

Isaac

API REST, validaciones, errores y demo en vivo

De qué habla

- **Endpoints.** Tres controladores con CRUD completo (GET, POST, PUT, DELETE) y los códigos de respuesta 200, 201 y 204.
- **DTOs y validaciones.** Por qué no se exponen las entidades, y cómo @Valid revisa las reglas del DTO (@NotBlank, @Email, @Past, @Positive).
- **Manejo de errores.** @RestControllerAdvice convierte las excepciones en 404, 409 y 400 con un formato de respuesta único.
- **Demo en vivo.** Crear datos, matricular, curso sin cupo, cancelar y liberar cupo, y los errores 400 y 404.
- **Conclusión.** Lo que se construyó, lo que aprendimos y las mejoras a futuro.

Qué muestra

StudentController.java · CreateStudentDto.java · GlobalExceptionHandler.java · Postman

Ronda de preguntas

Al terminar la demo se abre el espacio de preguntas. Cada integrante responde las preguntas sobre su parte.